

Partial Differential Equations 3 (Numerical)

Lecture 4 - Academic Year 2025/2026

Prof Halim Kusumaatmaja
Email: halim.kusumaatmaja@ed.ac.uk
Semester 2, Edinburgh Engineering

Aims and Objectives

1. To describe the matrix method.
2. Introduction to sparse matrix solvers.
3. Introduction to pre-conditioning.

Matrix Method: Setting Up

Recall in Lecture 2 that we have written the 2D Laplace equation in discretised form:

$$u_{xx} + u_{yy} = \frac{u_E - 2u_O + u_W}{\Delta x^2} + \frac{u_N - 2u_O + u_S}{\Delta y^2} = 0. \quad (1)$$

Setting $\Delta x = \Delta y$ for simplicity, we can rewrite this as

$$u_E + u_W - 4u_O + u_N + u_S = 0. \quad (2)$$

The idea in the matrix method is to write this as a system of linear, simultaneous equations. To do this, it is much more convenient to move away from a 2D indexing (i, j) to describe our 2D grid points to a 1D indexing k . This transformation is illustrated in Fig. 1 for any grid point and its corresponding east, west, north and south neighbours. Note that we have used python style counter where the counter starts from zero.

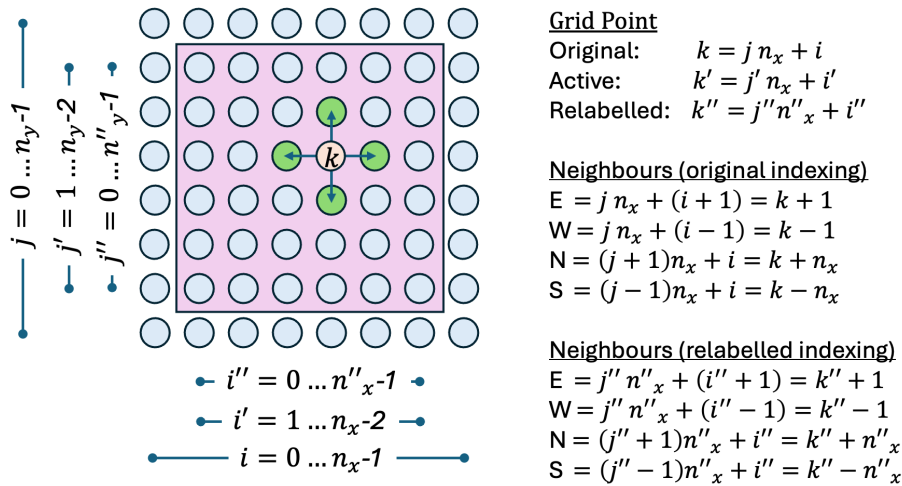


Figure 1: Labelling grid point and its neighbours with k -index. For convenience, we have also coloured the 'bulk nodes' (active nodes in the domain) in purple, while the outermost nodes are boundary nodes.

Now you may wonder why this whole transformation is useful. This is because in matrix method we will write the simultaneous equation in the matrix form of

$$\mathbf{A}\mathbf{u} = \mathbf{b}; \quad \text{or} \quad A_{kl}u_l = b_k, \quad (3)$$

where we have used the Einstein notation for repeating indices for the latter format. We want to use one index to denote the grid of interest and another index to denote the relevant neighbours. If we have kept our 2D indexing for our 2D grid, we would need a 4D matrix equation (and 9D matrix equation for 3D computations). This is cumbersome and not practical.

Exercise 1: Write the k -index (for the grid points and its neighbours) for a 3D grid where we have $i = 0 \cdots (n_x - 1)$, $j = 0 \cdots (n_y - 1)$, and $h = 0 \cdots (n_z - 1)$.

Let us now think about how to fill in the $\mathbf{A}$ matrix, and for simplicity let us think about the bulk/active nodes for now so that we do not have to worry about going out of bounds with our indices. It is helpful to write the discretised 2D Laplace equation in the notation of our k -index:

$$u_{k+1} + u_{k-1} - 4u_k + u_{k+n_x} + u_{k-n_x} = 0, \quad (4)$$

and compare it with the matrix form:

$$A_{k,k+1}u_{k+1} + A_{k,k-1}u_{k-1} + A_{k,k}u_k + A_{k,k+n_x}u_{k+n_x} + A_{k,k-n_x}u_{k-n_x} = b_k. \quad (5)$$

We can clearly spot that $b_k = 0$, $A_{k,k} = -4$, and $A_{k,k+1} = A_{k,k-1} = A_{k,k+n_x} = A_{k,k-n_x} = 1$.

Matrix Method: Boundary Condition

To complete the $\mathbf{A}$ matrix, we need to consider what to do at the boundaries. For simplicity, consider Dirichlet boundary condition. In this case, referring to Fig. 1, the outermost grid points have fixed values and we do not need to evolve them. Thereby, it is better if our computational domain only involves the ‘bulk nodes’ (i.e., $i' = 1 \cdots n_x - 2$ and $j' = 1 \cdots n_y - 2$, see figure). However, the ‘bulk nodes’ that interface with the boundary nodes need special treatment. For example:

- On the right interface, the east neighbour of a grid point lies outside the computational domain ($i = n_x - 1$ is beyond $i' = 1 \cdots n_x - 2$). u_E (at $i = n_x - 1$) is a fixed, known boundary value. Hence, we can re-write the discretised Laplace equation

$$u_W - 4u_O + u_N + u_S = -u_E \quad (6)$$

in a matrix form

$$A_{k,k+1}u_{k+1} + A_{k,k-1}u_{k-1} + A_{k,k}u_k + A_{k,k+n_x}u_{k+n_x} + A_{k,k-n_x}u_{k-n_x} = b_k, \quad (7)$$

with $b_k = -u_E$, $A_{k,k+1} = 0$, $A_{k,k} = -4$, and $A_{k,k-1} = A_{k,k+n_x} = A_{k,k-n_x} = 1$.

- Similar treatment is required for the west neighbour on the left interface, the north neighbour of the top interface, and the bottom neighbour of the south interface.
- At the southeast corner of the computational domain, both u_E and u_S are outside the domain and have fixed values. Here we write

$$A_{k,k+1}u_{k+1} + A_{k,k-1}u_{k-1} + A_{k,k}u_k + A_{k,k+n_x}u_{k+n_x} + A_{k,k-n_x}u_{k-n_x} = b_k, \quad (8)$$

with $b_k = -u_E - u_S$, $A_{k,k+1} = A_{k,k-n_x} = 0$, $A_{k,k} = -4$, and $A_{k,k-1} = A_{k,k+n_x} = 1$.

- Similar treatment is required for the other three corners.

A possible point of confusion: It is useful to reflect on our ‘indexing gymnastics’ at this point. We start with $i = 0 \cdots (n_x - 1)$ and $j = 0 \cdots (n_y - 1)$ when setting up the u_{ij} grid, leading to $k = 0 \cdots (n_x \times n_y - 1)$. However, our active computational domain is only $i' = 1 \cdots (n_x - 2)$ and $j' = 1 \cdots (n_y - 2)$. We can in principle continue the computation this way, but this wastes memory and CPU as there empty entries in our matrix elements. It is then customary to only store the bulk/active computational nodes such that $n''_x = (n_x - 2)$ and $n''_y = (n_y - 2)$, and shift the index to $i'' = 0 \cdots (n''_x - 1)$ and $j'' = 0 \cdots (n''_y - 1)$ with the new k -index now running as $k'' = j''n''_x + i''$.

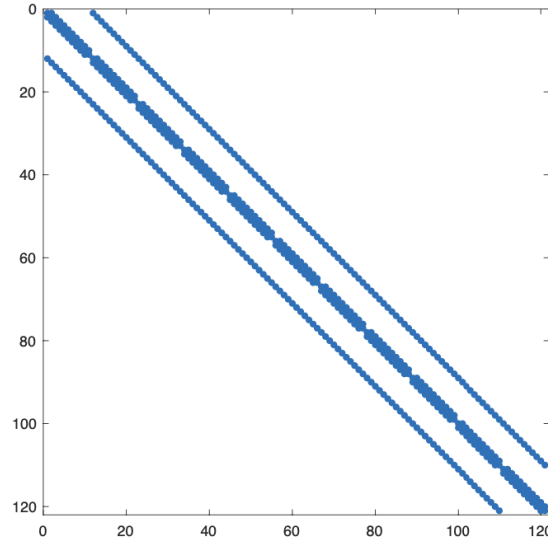


Figure 2: Example structure of the matrix $\mathbf{A}$.

The matplotlib `spy()` function will plot the non-zero values in a matrix, as illustrated in Fig. 2. Here we plot a computationally active 11×11 domain. This corresponds to a 13×13 grid when including boundary nodes.

Exercise 2: How should the coefficients of the $\mathbf{A}$ matrix be changed if we have Neumann boundary condition?

Example Problem

Consider the problem of a 200 mm square metal plate, which has a constant temperature gradient of between 0° and 50° Celsius along the top and right hand edges. The steady state temperature over the plate is found by solving

$$T_{xx} + T_{yy} = 0$$

subject to the boundary conditions

$$\begin{aligned} T(x, 0) &= 0 & 0.0 \leq x \leq 0.2 \\ T(0, y) &= 0 & 0.0 \leq y \leq 0.2 \\ T(x, 0.2) &= 250x & 0.0 \leq x \leq 0.2 \\ T(0.2, y) &= 250y & 0.0 \leq y \leq 0.2 \end{aligned} \quad (9)$$

We can solve $\mathbf{AT} = \mathbf{b}$ using Gaussian elimination. For example in python we use:

```
import scipy.sparse.linalg as LA
T = LA.inv(A) @ b
```

Fig. 3 illustrates the solution to this problem. This takes 14.2 s for a 101×101 grid. For large matrices this is very time consuming as it takes $(n_x n_y)^3$ arithmetic operations. For realistic PDE problems, direct method is not viable.

Sparse Matrix Method

You may already notice that the matrix $\mathbf{A}$ is sparse, meaning most of the matrix elements are zero. There are established methods to solve systems of linear equations for sparse matrices more effectively, known as the so-called Krylov subspace method. Since they are quite complex to implement, we revert to using available library routines. Table I tabulate available methods under python `scipy.sparse.linalg` as well as their time to solve our example problem for a 13×13 grid. For example, the relevant python snippet if using the `bicg` method is:

```
start = time.process_time()
```

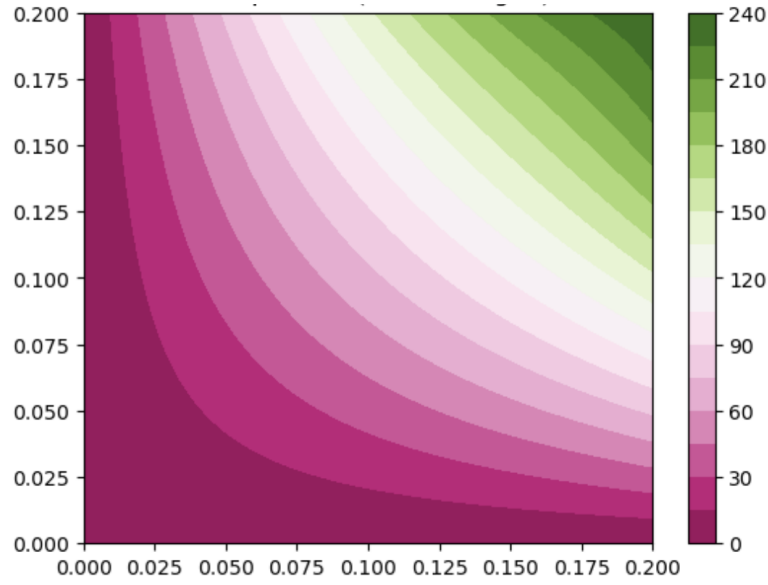


Figure 3: Solution to the example problem with boundary conditions in Eq. (9).

Table 1: Available sparse matrix solver in python.

Method	Run Time (s) on 13×13 grid
bicg: BIConjugate Gradient iteration	0.0126 s
bicgstab: BIConjugate Gradient STABilized iteration	0.0081 s
cg: Conjugate Gradient iteration	0.0528 s
cgs: Conjugate Gradient Squared iteration	0.0101 s
gmres: Generalized Minimal RESidual iteratio	0.0124 s
lgmres: Linear Solver restarted GMRES	0.0153 s
minres: Minimum Residual iteration	failed
qmr: Quasi-Minimum Residual iteration	0.0127 s
gcrotmk: GCROT algorithm	0.0099 s

```
T, info = LA.bicg(A,b,tol=0.5e-12)
stop = time.process_time()
```

Based on this trial, bicgstab performs the fastest and hence we will use this in our computing workshop 3, when we explore matrix methods. Using bicgstab we can also solve the example problem on a 101×101 grid. The result is obtained after for 0.7 s which is much faster than solving using direct matrix inversion in the previous section.

Pre-conditioning

Instead of solving the matrix equation $\mathbf{AT} = \mathbf{b}$ we can manipulate the matrix equation and ‘pre-condition’ it so that it is easier to solve. For example, we can write

$$\mathbf{AP}^{-1}\mathbf{PT} = \mathbf{b}. \quad (10)$$

Then, we can solve the equation in two steps: firstly solving $\mathbf{AP}^{-1}\mathbf{y} = \mathbf{b}$ for $\mathbf{y}$ and then solving $\mathbf{PT} = \mathbf{y}$ for $\mathbf{T}$. Careful choosing of $\mathbf{P}$ can lead to a matrix system which converges much faster than the un-conditioned system. Although work must be done to find suitable $\mathbf{P}$ and $\mathbf{P}^{-1}$, and to do the additional matrix multiplications, the overall solution time can be faster. Pre-conditioning is beyond the scope of our course, but this is something to consider if you try to solve matrix equation in the future.

Additional Materials

- The details are beyond our course, but if you are interested in the algorithm behind the bicgstab method, the wikipedia article is a good start:
https://en.wikipedia.org/wiki/Biconjugate_gradient_stabilized_method. For this course it is fine to just use the available library on python.

Version Control:

- version 1.0 on 20 December 2025